

AI-ENABLED DIGITAL TWIN SYSTEM

Health Monitoring, Fault Prediction & Mission Reliability for MALE UAV Piston Engines

CONFIDENTIAL - INTERNAL PLANNING DOCUMENT

Project Understanding & Technical Build Plan

This document has two parts:

Part 1 - What we are building, explained in simple language.

Part 2 - How we will build it, step by step (technical plan).

PART 1 - UNDERSTANDING THE PROJECT (IN SIMPLE LANGUAGE)

1.1 What is a MALE UAV?

- UAV = Unmanned Aerial Vehicle - a drone with no pilot on board.
- MALE = Medium Altitude, Long Endurance - it is built to fly for many hours at a time.
- It is used for surveillance, reconnaissance, communication relay, and defence missions.
- It flies using a piston engine - mechanically similar to a car or bike engine, just built for aircraft.

1.2 What Problem Are We Solving?

Think of a car's engine warning light - today's UAV systems work the same way: they only turn on the warning light after something has already gone wrong. There is no way to know an engine is about to fail before it actually happens.

For a UAV flying far from base, an engine failure mid-flight is serious: the mission fails, the drone can be lost, or it has to make an unsafe emergency landing. Our project aims to give an early warning, well before failure happens.

1.3 What is a "Digital Twin"?

A Digital Twin is a live virtual copy of the real engine - like a video-game version of it running on a screen, updated continuously with real engine readings. Anything happening to the real engine is mirrored on this virtual copy in real time.

This virtual copy lets engineers watch, test, and predict engine behaviour without needing to physically inspect the real engine every time.

1.4 What Will Our System Actually Do?

- Continuously watch the engine's vital signs (temperature, pressure, vibration, etc.).
- Notice early warning signs a human might miss.
- Warn the team before a failure happens - not after.
- Estimate how much healthy life the engine has left.
- Let the team replay any past flight to study what happened.
- Show all of this on one simple, easy-to-read screen.

PART 2 - TECHNICAL PLAN: HOW WE BUILD THE WORKING PROTOTYPE

We don't have access to a real UAV engine, so the prototype is built as a 7-step pipeline - starting with our own physics-based data, and ending with a live dashboard. Each step below has a Goal (what we're achieving) and How (the technical approach).

STEP 1 Create Our Own Engine Data (Physics-Based Simulation)

Goal: Since no real sensor data is available, generate realistic engine data ourselves so our AI models have real degradation patterns to learn from.

How we build it:

- Decide a mission profile first - idle, take-off, climb, cruise, descent - each with a target engine load.
- Use simplified physics formulas to turn that load into realistic sensor readings (see table below).
- Add small random noise to every reading, exactly like a real sensor would have.
- To create "failure" data: slowly drift one or more readings over time (e.g. rising CHT, rising vibration, falling oil pressure) until a simulated failure point - this also gives us the true Remaining Useful Life (RUL) at every moment, since we know exactly when failure happens in our own simulation.

Parameter	Depends On	Simplified Formula
RPM (engine speed)	Flight phase: idle, climb, cruise, descent	$\text{target_RPM}(\text{phase}) + \text{small_random_noise}$
Fuel Flow	RPM and throttle position	$k1 * \text{RPM} * \text{throttle_percent}$
Exhaust Gas Temp (EGT)	RPM and fuel flow	$\text{base_temp} + k2 * \text{RPM} + k3 * \text{fuel_flow} + \text{noise}$
Cylinder Head Temp (CHT)	Ambient temp, engine load, airspeed cooling	$\text{ambient_temp} + k4 * \text{load} - k5 * \text{airspeed} + \text{noise}$
Oil Pressure	RPM and oil temperature	$\text{base_pressure} + k6 * \text{RPM} - k7 * \text{oil_temp}$
Vibration	RPM (firing frequency)	$\text{sine_wave}(\text{firing_freq}) + \text{noise} (+ \text{spike if fault})$

This gives us a realistic, fully labelled dataset - normal running data, slowly degrading data, and the exact failure point - which is exactly what the AI/ML models in Step 5 need to learn from.

STEP 2 Build the Digital Twin Core

Goal: Create a live virtual model of the engine that stays in sync with the (simulated) data stream.

How we build it:

- Build a data pipeline that continuously reads the generated engine data, one time-step at a time.
- Keep a running "current state" of the engine, updated every few seconds - this is the virtual twin.
- Design it in independent parts (modular), so a real sensor feed could replace the simulator later.

STEP 3 Track Engine Health (Health Monitoring)

Goal: Continuously watch every key engine vital sign, not just react when something breaks.

How we build it:

- Store RPM, CHT, EGT, oil pressure & temperature, fuel flow, vibration, and battery/alternator health as an ongoing timeline.
- Define a normal healthy range for each parameter, used as the baseline for comparison.

STEP 4 Detect Faults Early

Goal: Catch problems - misfires, overheating, lubrication issues - while they are still small.

How we build it:

- Use simple threshold rules for obvious problems (e.g. CHT above a fixed limit).
- Use a trained model for subtler issues that a fixed threshold would miss, using the fault patterns from Step 1's data.

STEP 5 Predict the Future (AI/ML Layer)

Goal: Spot unusual behaviour early, and estimate how much healthy life the engine has left.

How we build it:

- Anomaly detection model - flags "this doesn't look like normal behaviour" (e.g. Isolation Forest).
- RUL (Remaining Useful Life) model - estimates cycles/hours remaining before failure, trained on the degrading data generated in Step 1 (the same idea NASA uses for its aircraft engine benchmark, adapted here to a piston engine).

STEP 6 Simulate Missions & Replay

Goal: Test the engine's behaviour under different conditions, and review any past flight.

How we build it:

- Let the user pick a scenario - hot weather, high altitude, rapid throttle changes - which changes the inputs to the Step 1 formulas.
- Store every simulated run so it can be replayed later, step by step, on the dashboard.

STEP 7 Show Everything on a Dashboard

Goal: Give the operator one clear screen to check engine health at a glance.

How we build it:

- Live charts for each tracked parameter.
- Colour-coded alerts - green (normal), yellow (watch), red (act now).
- A maintenance advisory panel and a mission-wise health report.

What We Deliver at the End

- A working software prototype/demo.
- The Digital Twin architecture design.
- The engine simulation model (Step 1).
- The AI/ML anomaly detection + RUL module.
- The visualisation dashboard.
- Technical documentation and a deployment roadmap.